Micro-Routing with Graph Neural Cellular Automata for Sub-Millisecond Agent Tool Selection

Julian M. Kleber
Independent Researcher
julian.m.kleber@gmail.com

November 2026

Abstract

Agent harnesses choose the next tool on every turn, usually with a multi-billion-parameter language model. We study a small learned "System-1" router for this decision. A Graph Neural Cellular Automaton (GNCA) holds the goal and the last four tool steps in five cells. It runs $K{=}4$ weight-tied message-passing steps over a fixed or learned graph and reads the next tool from the final cell states. We derive the update rule from random-walk diffusion on the trace graph, give its exact parameter count, and state which properties are proved. We test on four benchmarks (Hermes agent traces, BFCL, ToolBench, API-Bank; 5,127 held-out turns; 3 seeds). The 59,182-parameter tree GNCA is ahead of a Random Forest by +2.55 pp (random-effects pooled estimate, 95% CI [+0.67, +4.43]; +2.60 pp when the forest sees the same cell features). It shows no significant difference from HistGradientBoosting (+0.75 pp, CI [−1.50, +3.00]) or from a same-size MLP (−0.26 pp). On one CPU core it predicts in 0.403 ms (median), against 0.532 ms and 1.209 ms for the two tree ensembles. Lean 4 proofs (28 theorems, no `sorry`) cover receptive-field separation between graph topologies and the fact that K gated steps on a tree evaluate any decision tree of depth at most K. They do not certify the trained network: its per-step Jacobian has spectral norm 2.428 on average, so the recurrence is not contractive. The entropy-plus-step-norm escalation score detects routing errors no better than entropy alone (AUROC 0.730 vs. 0.732). Every number in the paper is generated from result files in the public repository.

1. Introduction

An autonomous agent works in a loop: it reads feedback from its environment (compiler output, file diffs, test logs) and picks the next action. Most of these picks are routine. Nervous systems handle routine motor actions with fast reflex arcs instead of deliberation; agent harnesses, in contrast, send every turn through a large autoregressive model. That costs latency and memory traffic on every turn and fills the context window with tool schemas. On our hardware two local models need 378 ms (Qwen3.5-9B) and 848 ms (Gemma-4-12B) per zero-shot routing decision on the Hermes test split. They reach only 35.01% and 38.02% top-1, below the majority-class rate of 52.86%.

A small supervised router trained on the harness's own traces is the obvious alternative, with the large model kept as a fallback for uncertain turns. Neural cellular automata (NCAs) [1] apply one learned local rule repeatedly; graph variants [2,3] run it on arbitrary graphs. An agent trace has natural cells (the goal and recent steps) and a small graph that can encode how the goal reaches the history, so a GNCA is a plausible router: tiny, weight-tied, and analysable. Earlier NCAs for text [6] work on 1D chains, where the goal needs $V{-}1$ steps to reach the latest cell.

Contributions.

- A GNCA next-tool router over five trace cells with chain, star, tree and learned topologies, with the update rule derived from random-walk diffusion, its spectrum and its exact parameter count (Section 3).
- Lean 4 formalisations of the structural facts behind the design, with an axiom audit and an explicit statement of what they do not cover; an empirical Jacobian check of the contraction hypothesis (Section 4).
- A four-benchmark, three-seed comparison against logistic regression, Random Forest, HistGradientBoosting (also on the same cell features as the GNCA) and a same-size MLP. Paired tests resample test turns with seeds kept together, use Holm correction and add a random-effects pooled estimate (Sections 5–6).
- Measured CPU latency and model size, a factorial study of depth, topology and loss, and a held-out test of the escalation score.

The headline is modest. The GNCA is a competitive small router. It is ahead of a Random Forest, shows no significant difference from gradient boosting or a same-size MLP (Table 3), and on one CPU core it predicts faster than both tree ensembles (Section 6). Version 7.6 of this manuscript claimed more than the result files support; Appendix C lists what was removed or replaced.

Scope. All claims concern turn-level next-tool accuracy on held-out splits. The primary model (tree GNCA, $K{=}4$, label smoothing) and the twelve primary comparisons are those coded in the analysis script. There is no pre-registration, and earlier exploratory runs on Hermes informed these choices, so the tests are not confirmatory

in the strict sense. Code, splits, result files, the scripts that generate every number, table and figure, and the Lean proofs are public (MIT licence).

2. Related Work

Neural cellular automata. Growing NCAs [1] learn local update rules on image grids that self-organise and repair. Learning Graph Cellular Automata [2] move the rule to arbitrary graphs and fit it to reproduce given dynamics (e.g. Voronoi or flocking rules); E(n)-equivariant GNCAs [3] add geometric equivariance for point clouds. NCAs have also been used for reasoning tasks [4], brain-inspired wiring [5] and language modelling [6,7]. Our cell rule is a standard GNCA update with the NCA perception stack (identity, neighbourhood average, Laplacian). What is specific here is the cell layout over an agent trace, the end-to-end classification objective on a fixed five-node graph whose nodes are trace events rather than points in space, and the evaluation.

Neural and soft decision trees. Deep neural decision forests [8], distilled soft trees [9] and deep neural decision trees (DNDT) [10] make tree routing differentiable. DNDT builds leaves from Kronecker products of soft bins, so its size grows exponentially with the number of features. Our Lean result about trees (Theorem 2) is a simulation statement in the same spirit: one shared cellular step on a tree evaluates any decision tree in depth-many steps.

Routing in LLM systems. FrugalGPT [11] and RouteLLM [12] route a whole query between models of different cost. AgentWeave [13] routes before reasoning in function calling, and recent work evaluates System-1 decision models for agent harnesses with paired, self-audited tests [14]. We address a narrower problem: predicting the next tool from the trace inside the loop, with an optional hand-off to a larger model when the router is unsure.

Verification of neural networks. SMT- and bound-propagation verifiers such as Marabou [21] and Beta-CROWN [22] certify input-output properties of trained feed-forward networks. Our Lean proofs are of a different kind: they establish properties of idealised graph automata, not of trained weights (Section 4).

Benchmarks. We use Berkeley Function Calling Leaderboard data [16] from the Gorilla project [15], ToolBench trajectories [17], API-Bank dialogues [18] and public Hermes agent traces [19].

3. Task Formulation & Architecture

Operational context and action. At turn t the agent sees a context $\mathbf{s}_t = (\mathbf{g}, \mathcal{H}_t)$. It consists of the user goal $\mathbf{g}$ and the trace $\mathcal{H}_t = \{(u_{t-k}, r_{t-k})\}_{k=1}^{4}$ of the last four tool calls u with their results r. The router predicts the next tool $y \in \mathcal{C}$ from a shared vocabulary of $|\mathcal{C}|$=13 tools (file read/edit, shell, search, respond, ...); a benchmark uses the subset that occurs in it. It also returns an escalation score s_{esc} (Section 3.4) that decides whether the turn is handed to a large model. Argument filling is left to the downstream model.

3.1. Trace graph and cell features

The graph has V=5 cells. Cell 0 holds the goal; cells $1, \dots, 4$ hold the last four steps, right-aligned so that cell 4 is the most recent step $t-1$ (cells without a step stay zero). Each cell carries D=146 observation channels:

- channels 1–128: a TF-IDF/truncated-SVD embedding fitted on the training goals of each benchmark and seed. Cell 0 embeds the goal text; step cells embed the text "tool + result" in the same basis;
- channels 129–132: four scalars. Goal cell: number of steps, log length of the last result, last-result error flag, log goal length. Step cells: position, log result length, error flag (result contains "error" or "fail"), active flag;
- channels 133–145: one-hot of the tool called in that step (zero for the goal cell);
- channel 146: occupancy mask.

Each cell also carries a hidden state $h_i \in \mathbb{R}^H$, H=16, initialised to zero, so the full cell vector has $C = D+H = 162$ channels.

3.2. Topologies and the random-walk operator

Let $S \in \{0,1\}^{V \times V}$ be a symmetric 0/1 pattern with self-loops ($S_{ii} = 1$) and $\Delta = \operatorname{diag}(S\mathbf{1})$ its degree matrix. The code uses the row-normalised operator

$$A = \Delta^{-1} S, \qquad A_{ij} \geq 0, \quad \textstyle\sum_j A_{ij} = 1. \tag{1}$$

Chain: $S_{ij} = 1$ for $|i-j| \leq 1$. *Star*: cell 0 linked to every cell. *Tree*: cell $i > 0$ linked to its parent $\lfloor (i-1)/2 \rfloor$, which gives

$$A_{\text{tree}} = \begin{pmatrix} 1/3 & 1/3 & 1/3 & 0 & 0 \\ 1/4 & 1/4 & 0 & 1/4 & 1/4 \\ 1/2 & 0 & 1/2 & 0 & 0 \\ 0 & 1/2 & 0 & 1/2 & 0 \\ 0 & 1/2 & 0 & 0 & 1/2 \end{pmatrix}, \tag{2}$$

so the two most recent steps are siblings under $t-4$. *Learned*: $A = \operatorname{softmax}_{\text{row}}(\alpha/\tau)$ with free logits $\alpha \in \mathbb{R}^{5\times5}$ initialised at $\log(A_{\text{tree}} + 10^{-4})$ and τ=0.5.

Sign convention and spectrum. We write $L_{\text{rw}} = I - A$ for the random-walk Laplacian [20]. We use the drift operator $-L_{\text{rw}} = A - I$; its i-th row is $\sum_j A_{ij}(x_j - x_i)$, the deviation of the neighbourhood mean from cell i.

Proposition 1 (Spectrum of the fixed operators)**.** *Let A be as in Eq. (1). (i) The eigenvalues of A are real and lie in $[-1, 1]$, with $A\mathbf{1} = \mathbf{1}$. (ii) The eigenvalues of $A - I$ are real and lie in $[-2, 0]$. (iii) On a connected*

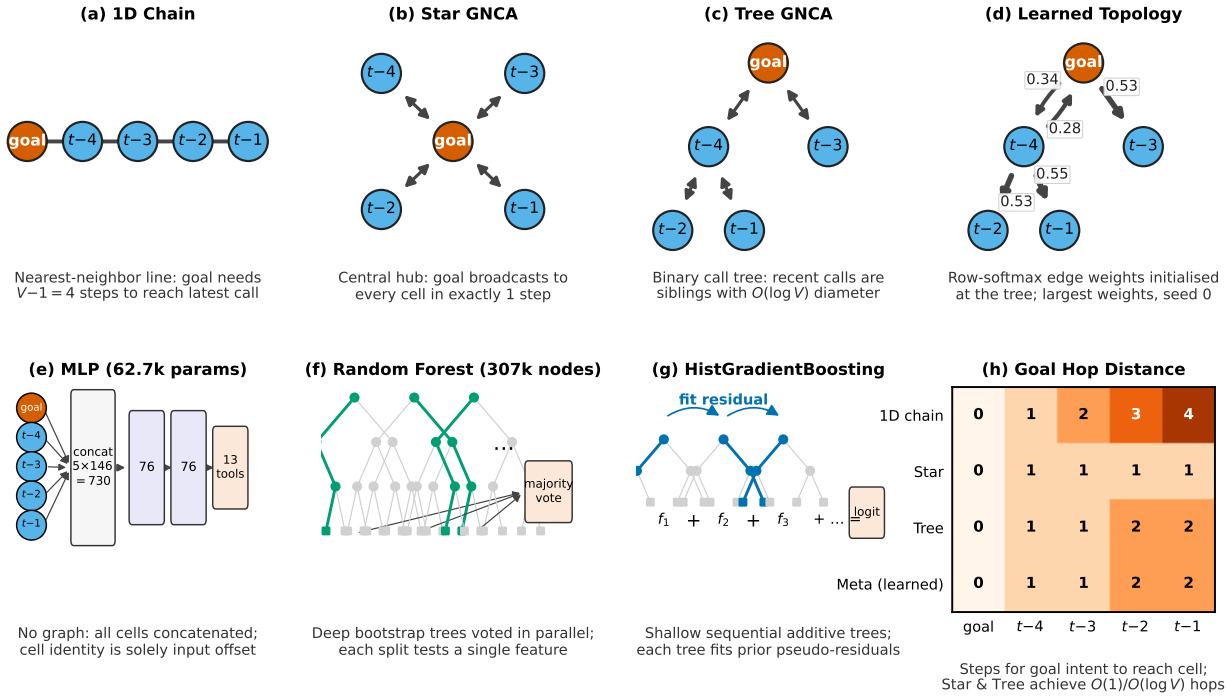


Figure 1: Information-flow graphs over five cells (goal cell and the last four steps $t-4, \ldots, t-1$), drawn from the adjacency matrices built by the training code, with the baselines for reference. (a)–(c) Fixed topologies; edges are undirected. (d) Learned topology: largest row-softmax weights of seed 0. (e)–(g) MLP, Random Forest and HistGradientBoosting baselines (parameter and node counts from the result files). (h) Number of steps the goal needs to reach each cell.

pattern, 0 *is a simple eigenvalue of* $A-I$ *with eigenvector* $\mathbf{1}$ *(consensus).*

Proof. $A = \Delta^{-1}S$ is similar to $\Delta^{-1/2}S\Delta^{-1/2}$, which is symmetric, so the spectrum of A is real. Row-stochasticity gives $\|A\|_\infty = 1$, hence $|\lambda| \le 1$, and $A\mathbf{1} = \mathbf{1}$. Shifting by $-I$ gives (ii). For (iii), A is the transition matrix of a random walk on a connected graph with self-loops, which is irreducible and aperiodic; by Perron–Frobenius the eigenvalue 1 is simple. □

For the matrices built by the code the second-largest eigenvalue of A is 0.833 (chain), 0.500 (star) and 0.766 (tree), and the smallest is −0.260, −0.300 and −0.280. A pure linear diffusion $x \leftarrow Ax$ therefore forgets non-consensus components fastest on the star; the learned update below is not restricted to diffusion.

Is an undirected graph a source of leakage? A common concern is that a graph operator over a sequence must be lower-triangular (causal) to avoid future-to-past contamination. For linear filters the general statement is true.

Proposition 2 (Leakage under linear filtering)**.** *Let* $X_1, \ldots, X_V$ *be scalar events in temporal order with innovations* $\eta_k = X_k - \mathbb{E}[X_k \mid X_{<k}]$ *that are independent of* $X_{<k}$ *and have* $\mathrm{Var}(\eta_k) = \sigma_k^2 > 0$*, and let* $\tilde{X} = MX$*. Then* $\tilde{X}_i$ *is a function of* $X_{\le i}$ *for every* i *if and only if* $M_{ik} = 0$ *for all* $k > i$*. If not, let* k *be the largest index with* $k > i$ *and* $M_{ik} \neq 0$*; then* $\mathrm{Cov}(\tilde{X}_i, \eta_k \mid X_{\le i}) = M_{ik}\sigma_k^2 \neq 0$.

Proof. $\tilde{X}_i = \sum_{j \le i} M_{ij}X_j + \sum_{j>i} M_{ij}X_j$. If the second sum vanishes, $\tilde{X}_i$ is measurable with respect to $X_{\le i}$. Otherwise write $X_k = \mathbb{E}[X_k \mid X_{<k}] + \eta_k$. Because k is the largest index with $M_{ik} \neq 0$, every other term of $\tilde{X}_i$ is a function of $X_{<k}$ and hence independent of η_k. Therefore $\mathrm{Cov}(\tilde{X}_i, \eta_k \mid X_{\le i}) = M_{ik}\,\mathrm{Var}(\eta_k) \neq 0$ and $\tilde{X}_i$ is not a function of $X_{\le i}$. A two-node example: with $S = \frac{1}{2}\mathbf{1}\mathbf{1}^\top$, $\tilde{X}_1 = \frac{1}{2}X_1 + \frac{1}{2}X_2$ carries $\frac{1}{2}\eta_2$. □

The proposition does not apply to our router. All five cells hold the goal and *past* steps that are observed when the prediction is made, and the label (the next tool) is in no cell. An undirected edge between $t-4$ and $t-1$ mixes two observed events; it carries no information from after the decision point. Directed masking would matter only if cells held events later than the decision, for example when training on full trajectories with a label at every step. We therefore use undirected graphs, as the code does, and make no causal-filtering claim.

3.3. Cellular update rule

From diffusion to the perception stack. The linear diffusion $\dot{\mathbf{x}} = (A - I)\mathbf{x}$ discretised with unit step gives

$\mathbf{x}^{(t+1)} = A\mathbf{x}^{(t)}$, i.e. repeated averaging. NCAs [1] instead pass the identity, a smoothing and a difference filter to a learned per-cell network. On the graph these are x, Ax and $(A-I)x = Ax - x$:

$$p_i^{(t)} = \left[\, x_i^{(t)},\ [A\mathbf{x}^{(t)}]_i,\ [(A-I)\mathbf{x}^{(t)}]_i \,\right] \in \mathbb{R}^{3C}, \quad (3)$$

with $x_i^{(t)} = [o_i;\, h_i^{(t)} + e_t]$, o_i the fixed observation of cell i and $e_t \in \mathbb{R}^H$ a learned step embedding. The third block is the difference of the other two, so it adds no information; we keep it because it is the standard NCA perception stack [1] and costs only weights.

Gated residual update. A shared two-layer network (a 1×1 convolution over cells) maps the perception to a candidate update and a gate,

$$\begin{aligned} [\delta_i;\, g_i] &= W_2\, \mathrm{ReLU}(W_1 p_i^{(t)} + b_1), \\ h_i^{(t+1)} &= h_i^{(t)} + \sigma(g_i) \odot \delta_i, \end{aligned} \quad (4)$$

with $W_1 \in \mathbb{R}^{96\times 3C}$ and $W_2 \in \mathbb{R}^{2H\times 96}$. W_2 is initialised at zero, so training starts from the identity map; σ is the logistic function. The rule is shared across cells and steps. After K steps the tool head reads $\hat{y} = \mathrm{softmax}(W_{\mathrm{tool}}\,\mathrm{vec}(\mathbf{h}^{(K)}) + b_{\mathrm{tool}})$ from the flattened 5×16 hidden states. There is no stochastic cell masking, damping factor or halting in the primary model; inference is deterministic.

Worked information flow on the tree (K=4). At step 1 the leaf cells $t-2$ and $t-1$ (results, error flags) update from their parent $t-4$ and themselves, and $t-4$ averages over the goal and both leaves. At step 2 the goal cell, which averages over $t-4$ and $t-3$, receives the leaves' information through $t-4$. Steps 3–4 send goal-conditioned state back to the leaves. The goal reaches every cell within two steps and every pair of cells is connected within three (Figure 1h); on the chain the goal needs four steps to reach $t-1$.

Parameter count. With D_{mid}=96 the module has

$$\begin{aligned} N &= \underbrace{96 \cdot 3C + 96}_{W_1, b_1} + \underbrace{2H \cdot 96}_{W_2} + \underbrace{KH}_{e_t} + \underbrace{VH|\mathcal{C}| + |\mathcal{C}|}_{\text{tool head}} + N_{\mathrm{aux}} \\ &= 46{,}752 + 3{,}072 + 64 + 1{,}053 + N_{\mathrm{aux}}, \end{aligned} \quad (5)$$

where $3C = 486$. The PyTorch module also contains a bilinear argument head ($32 \cdot H^2 + 32 = 8{,}224$) and an abstain gate ($H + 1 = 17$), together N_{aux}. The code counts them in its totals (N = 59,182 for K=4, 59,246 for K=8, 59,207 with learned logits). They receive no gradient, because the loss is cross-entropy on the tool head only. The trained part has 50,941 parameters. The weights do not depend on K except through e_t, and the history length is fixed by V.

3.4. Escalation score

For hand-off we score each turn with

$$s_{\mathrm{esc}} = \underbrace{\frac{\mathcal{H}(\hat{y})}{\ln|\mathcal{C}|}}_{\bar{\mathcal{H}} \in [0,1]} + \beta\, \underbrace{\frac{1}{V}\sum_{i=1}^{V} \frac{\|h_i^{(K)} - h_i^{(K-1)}\|_2}{\sqrt{H}}}_{\bar{s}}, \quad (6)$$

and escalate when $s_{\mathrm{esc}} > \theta$. The evaluation code uses β=1, fixed before evaluation and not tuned on a validation split. Section 6 reports the threshold-free AUROC and two operating points on held-out turns. The $1/\sqrt{H}$ factor makes $\bar{s}$ a per-coordinate root mean square. If a step difference $z \in \mathbb{R}^H$ has i.i.d. $\mathcal{N}(0, \sigma^2)$ coordinates, then $\|z\|_2/\sigma$ follows a χ_H distribution and

$$\mathbb{E}\|z\|_2 = \sigma\sqrt{2}\,\frac{\Gamma\left(\frac{H+1}{2}\right)}{\Gamma\left(\frac{H}{2}\right)} = \sigma\sqrt{H}\left(1 - \frac{1}{4H} + O(H^{-2})\right), \quad (7)$$

so $\mathbb{E}\,\bar{s} \approx \sigma$ independently of H and V. The entropy term is unitless; the step term carries the units of h, and rescaling h by $c > 0$ is the same as replacing β by $c\beta$. This normalization therefore removes the dependence on H and V but not on the scale of h, so β has no natural value. Section 6 tests the score on held-out turns.

4. Structural Properties & Formal Verification

The Lean 4 development (`formal_proofs/`, Lean 4.35.0, 28 top-level theorems) builds with `lake build` and contains no `sorry`. `#print axioms` shows that 13 theorems use no axioms and the rest use only Lean's standard axioms (`propext`, `Quot.sound`, `Classical.choice`); none depends on `sorryAx` or a user-declared axiom (Appendix A). Distances and weights in the Lean files are natural numbers, so the metric and contraction statements are about discrete abstractions of the real-valued model. Table 1 maps each group to what it does and does not say about the trained router; Appendix A lists every theorem by name.

Theorem 1 (Receptive-field separation; `theorem_receptive_field_separation`). *Let $V \geq 3$. Every walk from cell 0 to cell $V-1$ in the chain graph has length at least $V-1$. In the star graph with hub 0, every ordered pair of cells is joined by a walk of length 2.*

With self-loops, information from cell j can influence cell i after K steps of Eq. (4) only if a walk of length $\leq K$ joins them, because each step mixes only graph neighbours. With V=5 and K=4 every topology therefore reaches every cell; the theorem explains why the goal reaches the history in fewer steps on the star and tree, not that they are more accurate. Companion theorems show that the chain, star and binary call-tree patterns (parent$(i) = \lfloor (i-1)/2 \rfloor$) are symmetric adjacency relations admitted by the model class. Neighbourhoods and walks commute with node permutations (`neighborhood_equivariance`, `path_equivariance`).

Theorem 2 (Tree simulation; `decision_tree_cellular_duality`). *Let T be a binary decision tree with*

gates $g_u : \mathcal{X} \to \{0,1\}$ *at internal nodes and labels at leaves. Let* Φ_x *replace a tree whose root has gate* g *and children* (l, r) *by* l *if* $g(x) = 1$ *and by* r *otherwise; leaves are fixed points. Then: (i)* $\mathrm{eval}(\Phi_x(T), x) = \mathrm{eval}(T, x)$ *for every* T *and* x*. (ii) If* $\mathrm{depth}(T) \le K$*, then* $\Phi_x^K(T) = \mathrm{leaf}(\mathrm{eval}(T, x))$*. (iii) A complete tree of depth* K *has* $2^K - 1 \ge K$ *internal nodes. So* M *such trees with* d *parameters per split need* $M(2^K - 1)d$ *parameters, while one shared rule applied* K *times needs* d*.*

Proof sketch. (i) By cases on the root. For a leaf, Φ_x is the identity. For a branch, $\mathrm{eval}(T, x)$ is by definition eval of the child selected by $g(x)$, which is $\Phi_x(T)$. (ii) Induction on K. For K=0, $\mathrm{depth}(T) \le 0$ forces a leaf. For K+1, a leaf is fixed. A branch with depth $\le K$+1 steps to a child of depth $\le K$, and the induction hypothesis together with (i) gives the leaf $\mathrm{eval}(T, x)$. (iii) $2^K \ge K + 1$ by induction. □

This is a statement about an idealised discrete automaton: a tree-shaped cellular update *can* represent any depth-K decision tree with one shared rule, and K steps are enough. It does not say that gradient training finds such a rule, and the five-cell graph of Section 3.1 is not the decision tree itself. A further theorem (`sigmoidal_gating_*_margin_concordance`) states an elementary fact. A threshold at 1/2 on a sigmoid output agrees with the hard gate whenever the output is at least a margin $\delta > 0$ away from 1/2.

Theorem 3 (Conditional contraction; `fixed_point_unique`, `trajectory_decay`)**.** *Let* T *be an operator on a metric space with* $2\,d(Tx, Ty) \le d(x, y)$ *for all* x, y*. Then* T *has at most one fixed point, and* $d(T^n x, T^n y) \le 2^{-n} d(x, y)$*. The premise holds whenever* $b\,d(Tx, Ty) \le (r + \ell)\,d(x, y)$ *for natural numbers with* $2(r+\ell) \le b$ *(`trained_damped_recurrence_is_dyadic_contraction`).*

For a damped update $h \leftarrow (1 - \alpha)h + \alpha\Delta(h)$, r/b plays the role of the retention $1 - \alpha$ and ℓ/b bounds the Lipschitz constant of $\alpha\Delta$. The Lipschitz bound is a hypothesis. Appendix B derives the standard perturbation bound that would connect a contractive surrogate to a nearby operator. It reads $\limsup_t \|h^{(t)} - \tilde{h}^*\| \le \varepsilon/(1-\tilde{L})$, where $\tilde{h}^*$ is the fixed point of an operator $\tilde{T}$ with Lipschitz constant $\tilde{L} < 1$ and $\sup_h \|T(h) - \tilde{T}(h)\| \le \varepsilon$. It too needs $\tilde{L} < 1$.

Empirical check of the contraction hypothesis. The Jacobian of one step of Eq. (4) with respect to the 5×16 hidden state is $J = I + \partial(\sigma(g) \odot \delta)/\partial h$; at initialisation $W_2 = 0$ and $J = I$. We computed J for every Hermes test turn, every step and every seed of the trained tree GNCA (42,360 Jacobians). Its spectral norm is 2.428 on average, 2.872 at the 95th percentile and 3.336 at most; 0.0% are below 1. The ratio of successive hidden-state update norms of the K=8 tree model averages 1.20, so updates grow rather than shrink. The trained router is not a contraction, and neither Theorem 3 nor the perturbation bound applies to it. We also trained a variant with spectrally normalised W_1, W_2 and a bounded update $h \leftarrow h + 0.25\,\sigma(g) \odot \tanh(\delta)$. It reaches only 61.99% Hermes test top-1 (68.90% for the unconstrained tree model in the same run; Table 4). Note that the identity term keeps the Lipschitz constant of this variant at up to $1 + 0.25L$, so spectral normalisation alone does not satisfy the premise either. With only K=4 steps the router is well defined without contraction, so the failed premise does not affect the results; it does rule out any certified stability claim.

5. Experimental Protocol

Data. Hermes: next-tool turns from public Hermes agent traces [19], split by session into 57,675 training, 3,465 validation and 3,530 test turns. BFCL: all public BFCL v3 single-turn AST, live and irrelevance files [16], with labels mapped heuristically onto the tool vocabulary. Rows with identical goal text stay in one split, stratified 60/20/20 by label (2,083/698/692). BFCL items are single-turn, so the history cells are mostly empty there. ToolBench [17] and API-Bank [18]: session-disjoint 60/20/20 splits (1,324/452/422 and 1,552/517/483). Split manifests and preparation scripts are in the repository.

Models. We compare the majority class, logistic regression, a Random Forest (50 trees, maximum depth 15) and HistGradientBoosting (200 iterations, 31 leaves, learning rate 0.1). The MLP has two hidden layers of 76 units (LayerNorm, GELU, dropout 0.1) on the flattened 5×146 cell features (62,713 parameters). The GNCA uses chain, star, tree and learned topologies. The primary tree baselines receive 188 features: the goal embedding, a positional one-hot of the last four tools and four scalars, but not the per-step result embeddings. To remove this confound we also fit both tree models on exactly the 730 flattened cell features of the MLP and GNCA (RF*, HGB*). They use the same hyperparameters and seeds (source S1, 111 s on CPU). These form a secondary comparison family. All hyperparameters are fixed defaults; no method received a hyperparameter search.

Training. Neural models use AdamW (learning rate 2×10^{-3}, weight decay 0.05, label smoothing 0.05, batch 512, cosine schedule) for 800 steps (3,000 on Hermes) and keep the checkpoint with the best validation top-1. Seeds 0, 1, 2 change the SVD fit, initialisation and batch order. All runs used one NVIDIA RTX 3060.

Statistics. The primary model is the tree GNCA with K=4 and label smoothing, compared with Random Forest, HistGradientBoosting and MLP on each benchmark (12 comparisons). For each comparison we average each test turn's correctness over the three seeds and bootstrap

Table 1: Scope of the formal results. "Proved" means machine-checked in Lean 4 for the stated (idealised, natural-number) objects. No row certifies the trained weights.

Group	Theorems	Proved	Relation to the trained router
Receptive fields	11	Chain needs $\geq V-1$ steps from goal to last cell; star joins all pairs in 2; chain, star and call tree are admissible patterns	Exact for the fixed graphs used; says nothing about accuracy (topologies within about 1 pp, Table 4)
Equivariance	3	Neighbourhoods and walks commute with relabelling	Holds for the message-passing step; the cell order is fixed by the trace, so it is not exploited
Tree simulation	5	K shared gated steps evaluate any decision tree of depth $\leq K$	Representability only; training is not shown to find such a rule
Contraction	5	Dyadic contraction $\Rightarrow$ unique fixed point and 2^{-n} decay; scaled Lipschitz bound $\Rightarrow$ dyadic	Premise fails: Jacobian norm 2.428 on average, 0.0% below 1
Aggregation, gating	4	Convex weights preserve bounds; sigmoid threshold matches the hard gate outside a margin	Elementary facts used in the design

the mean paired difference over test turns (10,000 resamples). The resampling unit is thus the test turn, and n is the number of test turns, not seeds $\times$ turns. p-values are Holm-adjusted within the primary family (12) and the secondary family (8). Pooled effects use a DerSimonian–Laird random-effects model over the four benchmarks. We test differences, not equivalence: a non-significant difference is reported as such.

Hermes factorial. A separate Hermes run (source S2, 3 seeds, its own feature packing) varies depth $K \in \{2, 4, 8\}$, topology, the loss (label smoothing vs. class weights), cross-turn state carry, learned halting and the contractive variant. It also holds the escalation evaluation on the K=8 tree model. There β=1 and the thresholds $\theta \in \{0.50, 0.65, 0.76, 0.85, 1.00\}$ are fixed in the script, not tuned on a validation split; we report the threshold-free AUROC and two operating points on held-out test turns.

Latency. Batch-1 prediction on one thread of an AMD Ryzen 9 7950X 16-Core Processor, after 100 warm-up calls, median over 1,000 calls on Hermes test inputs. Tree models are timed through scikit-learn's `predict_proba` and the GNCA through a PyTorch forward pass, so both include framework overhead; featurisation is not included.

6. Results

Accuracy. Table 2 and Figure 2 give all models on all benchmarks. Every learned model is far above the majority class. The tree GNCA reaches 69.10% on Hermes, 76.40% on BFCL, 76.30% on ToolBench and 85.99% on API-Bank. No model wins everywhere: HistGradientBoosting is best on BFCL (78.18%), the MLP on Hermes (69.91%), and the star GNCA on API-Bank (87.09%). Choosing the GNCA topology on validation instead of fixing the tree would pick the star on Hermes (69.66% test) and the chain on BFCL (76.01%). It would pick the learned topology on ToolBench (75.75%) and the star on API-Bank (87.09%). We keep the pre-specified tree for all tests.

Paired tests. After Holm correction (Table 3, Figure 3) two of twelve primary comparisons are significant at 0.05. The tree GNCA beats HistGradientBoosting on Hermes (+2.90 pp, $p_{\text{Holm}} = 0.001$) and the Random Forest on API-Bank (+5.87 pp, $p_{\text{Holm}} = 0.002$). On BFCL its difference to HistGradientBoosting is −1.78 pp (CI $[-3.71, +0.14]$, not significant). Pooled over benchmarks, the GNCA is ahead of the Random Forest (+2.55 pp, CI $[+0.67, +4.43]$, $p = 0.008$). It shows no significant difference from HistGradientBoosting (+0.75 pp, between-benchmark variance $\tau^2 = 4.36$) or the MLP (−0.26 pp, CI $[-1.43, +0.92]$).

Input-matched tree models. Given the same cell features (lower block), the Random Forest closes the gap on Hermes (+0.07 pp, CI $[-0.81, +0.95]$) but remains below the GNCA on the other three benchmarks (significant after Holm on BFCL and API-Bank). Pooled, the GNCA is ahead by +2.60 pp (CI $[+0.12, +5.07]$, $p = 0.040$). Against HistGradientBoosting on cell features (Table 3) the pooled difference is +0.12 pp (CI $[-2.02, +2.25]$). The GNCA is ahead on Hermes (+2.29 pp, $p_{\text{Holm}} = 0.001$), with no significant difference on the other three. On BFCL the history cells are mostly empty. HistGradientBoosting reaches the same accuracy with either feature set, while the Random Forest, which samples a random subset of features at each split, loses accuracy on the longer, mostly zero cell vector.

Protocol sensitivity. An earlier Hermes run used a different feature pipeline for the tree models (source S3). It gave HistGradientBoosting 69.47%, Random Forest 67.88% and the star and tree GNCA 66.19% and 65.27%, the opposite ranking against gradient boosting. Which model wins on Hermes therefore depends on the features the tree models receive. On BFCL, training the GNCA for 150,000 steps instead of 800 (source S4) gives 75.29–76.40% across topologies, still below HistGradientBoosting (78.18%).

Depth, topology and loss. In the factorial run (Table 4) going from K=2 to K=4 helps (67.40% → 68.90%)

Table 2: Test top-1 accuracy (%, mean ± SD over seeds 0–2). Source S1 (Table 5). Tree-model parameter counts are storage counts: Random Forest nodes × (2 + classes), HistGradientBoosting nodes × 5. GNCA counts include the untrained auxiliary heads (Section 3.3).

Model	Params (Hermes)	Hermes $n_{\text{test}} = 3530$	BFCL (E1v2 split) $n_{\text{test}} = 692$	ToolBench $n_{\text{test}} = 422$	API-Bank $n_{\text{test}} = 483$
Majority class	1	52.86 ± 0.00	56.07 ± 0.00	36.26 ± 0.00	33.54 ± 0.00
Logistic regression	2,457	65.21 ± 0.12	74.52 ± 0.37	56.71 ± 0.49	79.23 ± 0.24
Random Forest	4,606,770	67.93 ± 0.31	74.18 ± 0.96	74.33 ± 1.12	80.13 ± 0.55
HistGradientBoosting	43,515	66.20 ± 0.32	78.18 ± 0.58	76.07 ± 1.24	84.75 ± 0.59
Random Forest, cell features	–	69.04 ± 0.35	72.64 ± 0.09	73.38 ± 0.59	81.64 ± 0.98
HistGradientBoosting, cell features	–	66.81 ± 0.70	78.18 ± 0.58	77.25 ± 0.48	85.64 ± 0.12
MLP (76 hidden)	62,713	69.91 ± 0.45	77.60 ± 0.15	74.17 ± 1.19	86.47 ± 0.73
GNCA chain	59,182	69.11 ± 0.16	76.01 ± 0.72	75.12 ± 0.63	85.09 ± 1.69
GNCA star	59,182	69.66 ± 0.18	76.30 ± 1.39	75.35 ± 1.85	87.09 ± 1.57
GNCA tree (primary)	59,182	69.10 ± 0.12	76.40 ± 0.67	76.30 ± 1.85	85.99 ± 1.38
GNCA learned	59,207	69.44 ± 0.30	76.83 ± 0.60	75.75 ± 1.07	86.75 ± 1.69

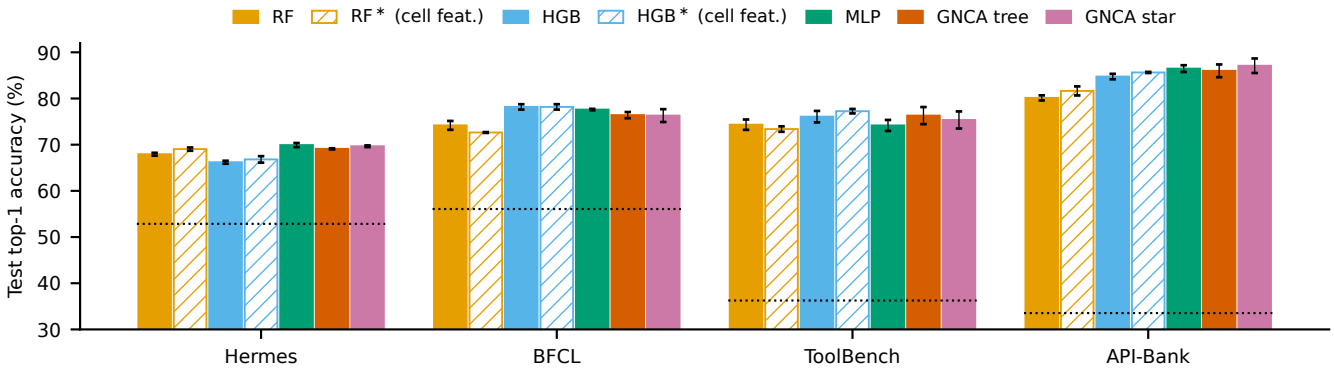


Figure 2: Test top-1 accuracy per benchmark (bars: mean over 3 seeds; whiskers: SD). Hatched bars: tree models fitted on the same cell features as the GNCA. Dotted lines: majority class.

and K=8 adds nothing (68.92%). The topologies span about one percentage point, from 67.88% (chain) to 68.90% (tree), with the star at 68.77% and the learned graph at 68.03%. This is consistent with Theorem 1: with K=4 every topology reaches all cells. Cross-turn state carry and learned halting do not help. On the learning curve, the best-so-far validation accuracy of the tree model (K=4) is 68.15% after 500 steps and 70.79% after 3,000. The gains at the intermediate checkpoints are gradual, with no sudden late jump. We therefore see no delayed-generalisation ("grokking") phase and dropped that term from the title.

Rare tools. Class-weighted training trades top-1 for macro-F1. On Hermes (Table 2 run) the class-weighted tree GNCA has macro-F1 43.02% at 65.46% top-1, against 35.08% at 69.10% with label smoothing. Random Forest and HistGradientBoosting reach 28.46% and 31.56% macro-F1, the MLP 39.74%.

CPU cost. On one core the tree GNCA predicts in 0.403 ms (95th percentile 0.461 ms), HistGradientBoosting in 0.532 ms and the Random Forest in 1.209 ms, so the GNCA is 1.3× and 3.0× faster. Serialised sizes are 240 kB (GNCA weights), 0.78 MB (HistGradientBoosting) and 51.62 MB (Random Forest, 307,118 nodes). All three are two to three orders of magnitude cheaper than the local LLMs in Section 1. The gap between the small models is small in absolute terms and matters only where routing runs on every turn under a tight budget.

Escalation score. On the Hermes test split (K=8 tree model, 3 seeds) s_{esc} separates wrong from correct routing decisions with AUROC 0.730 ± 0.008. Entropy alone gives 0.732 ± 0.001 and the step-norm term alone 0.441 ± 0.023, below chance: larger final updates go with *correct* predictions. At θ=0.76 the router escalates 11.4% of turns and is 72.45% accurate on the turns it keeps; at θ=0.50 it escalates 72.6% and is 90.48% accurate on the rest. The step-norm term adds nothing, and predictive entropy should be used on its own.

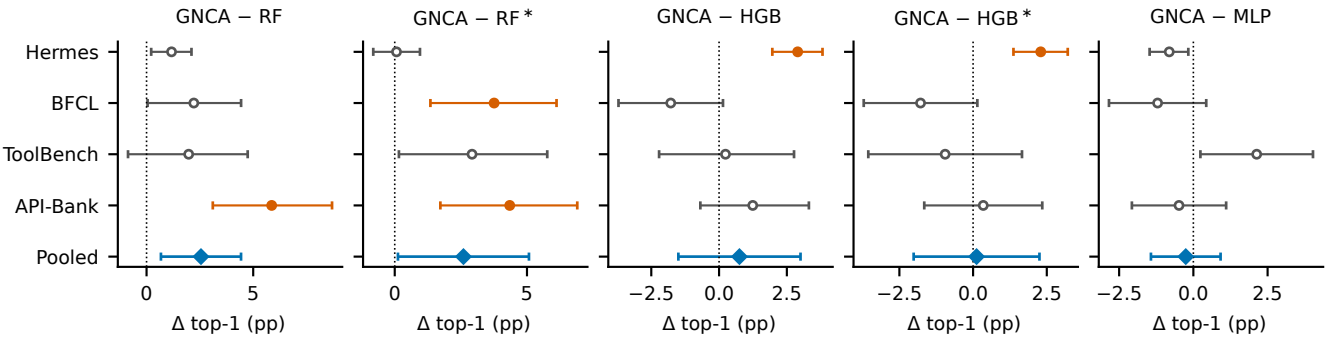


Figure 3: Paired differences from Table 3 with 95% bootstrap CIs. Filled orange: Holm-adjusted $p < 0.05$; open grey: not significant; blue diamonds: random-effects pooled estimate. Source S1.

Table 3: Paired comparisons, tree GNCA minus baseline (pp). Seed-clustered bootstrap over test turns. Upper block: primary family (Holm over 12); lower block: secondary family (Holm over 8). Pooled: random-effects estimate over four benchmarks. RF: Random Forest; HGB: HistGradientBoosting; *: fitted on the GNCA's cell features.

Benchmark	Baseline	Δ (pp)	95% CI	p	p_{Holm}
Hermes	RF	+1.17	[+0.22, +2.11]	0.0155	0.1530
Hermes	HGB	+2.90	[+1.96, +3.81]	0.0001	0.0012
Hermes	MLP	-0.81	[-1.47, -0.17]	0.0153	0.1530
BFCL	RF	+2.22	[+0.05, +4.43]	0.0515	0.3605
BFCL	HGB	-1.78	[-3.71, +0.14]	0.0670	0.4020
BFCL	MLP	-1.20	[-2.84, +0.43]	0.1505	0.7524
ToolBench	RF	+1.97	[-0.87, +4.74]	0.1794	0.7524
ToolBench	HGB	+0.24	[-2.21, +2.76]	0.8812	1.0000
ToolBench	MLP	+2.13	[+0.24, +4.03]	0.0307	0.2456
API-Bank	RF	+5.87	[+3.11, +8.70]	0.0002	0.0022
API-Bank	HGB	+1.24	[-0.69, +3.31]	0.2246	0.7524
API-Bank	MLP	-0.48	[-2.07, +1.10]	0.5507	1.0000
Pooled	RF	+2.55	[+0.67, +4.43]	0.0077	–
Pooled	HGB	+0.75	[-1.50, +3.00]	0.5132	–
Pooled	MLP	-0.26	[-1.43, +0.92]	0.6686	–
Secondary: tree models on the same cell features (Holm over 8)					
Hermes	RF*	+0.07	[-0.81, +0.95]	0.8969	1.0000
Hermes	HGB*	+2.29	[+1.37, +3.21]	0.0001	0.0008
BFCL	RF*	+3.76	[+1.35, +6.12]	0.0021	0.0126
BFCL	HGB*	-1.78	[-3.71, +0.14]	0.0670	0.2680
ToolBench	RF*	+2.92	[+0.16, +5.77]	0.0445	0.2225
ToolBench	HGB*	-0.95	[-3.55, +1.66]	0.4598	1.0000
API-Bank	RF*	+4.35	[+1.73, +6.90]	0.0014	0.0098
API-Bank	HGB*	+0.35	[-1.66, +2.35]	0.7301	1.0000
Pooled	RF*	+2.60	[+0.12, +5.07]	0.0398	–
Pooled	HGB*	+0.12	[-2.02, +2.25]	0.9139	–

7. Discussion and Limitations

Where the GNCA fits. The GNCA is not more accurate than a same-size MLP on the same inputs, and it shows no significant difference from gradient boosting (Table 3). Its practical case is cost: a single 240 kB rule runs in under half a millisecond on one core, is ahead of a 51.62 MB Random Forest when pooled, and is differentiable end to end. Its distinguishing parts (topology, weight-tied recurrence, step-norm uncertainty) did not translate into accuracy or better error detection in our runs.

Table 4: Hermes factorial (test top-1, %, mean ± SD over 3 seeds; source S2). Columns: training loss, label smoothing (LS) or class weights (CW). Carry: hidden state carried across turns; halting: learned halting. Contractive variant: spectrally normalised W_1, W_2, update $0.25\,\sigma(g) \odot \tanh(\delta)$.

Configuration	Params	LS	CW
Tree, K=2	59,150	67.40 ± 0.20	63.81 ± 0.71
Tree, K=4	59,182	68.90 ± 0.21	64.47 ± 0.73
Tree, K=8	59,246	68.92 ± 0.47	64.33 ± 0.18
Chain, K=4	59,182	67.88 ± 0.23	64.70 ± 0.34
Star, K=4	59,182	68.77 ± 0.24	64.63 ± 0.12
Learned, K=4	59,207	68.03 ± 0.36	64.35 ± 0.88
Tree, K=4, carry	59,182	67.55 ± 0.07	62.93 ± 0.78
Tree, K=4, halting	59,199	67.12 ± 0.26	63.41 ± 0.54
Tree, K=4, contractive	59,182	61.99 ± 0.14	63.23 ± 0.31

Limitations.

- **Novelty.** The cell rule is a standard GNCA update; the contribution is its use as a next-tool router and a careful evaluation, not a new primitive.
- **No hyperparameter search.** All models use fixed configurations; a tuned baseline could change the ranking, as the protocol-sensitivity result already shows for Hermes.
- **Primary tree baselines see fewer features.** The input-matched secondary family changes the Hermes Random Forest result but not the overall picture.
- **Three seeds, small test sets.** ToolBench, API-Bank and BFCL have 422–692 test turns; their primary CIs in Table 3 are between 3 and 6 pp wide.
- **Weak LLM baselines.** The two local LLMs ran zero-shot without tool-specific prompting or fine-tuning; they show cost, not the best LLM accuracy.
- **Turn-level evaluation only.** We measure next-tool accuracy, not task success in a running agent; earlier agent-loop pilots were too small to support conclusions and are not reported.
- **Fixed vocabulary and window.** Every benchmark is mapped onto 13 tools and a five-cell window. New tools (e.g. dynamically loaded MCP servers) need a

new head or an open-vocabulary scorer; longer histories need pooling or more cells. We did not test either.

- **Formal results are idealised.** Nothing is proved about trained weights, and the trained recurrence is not contractive.
- **One machine for latency,** including framework overhead and excluding featurisation.

Broader impact. A cheap router reduces the energy spent on routine turns, but faster loops can also propagate agent errors faster. Harnesses should keep permission sandboxes and human approval for irreversible actions regardless of the router.

8. Conclusion

A 59,182-parameter graph cellular automaton is a reasonable next-tool router. On four benchmarks it is ahead of a Random Forest and shows no significant difference from gradient boosting or a same-size MLP (Table 3; we did not test equivalence). It runs in well under a millisecond on one CPU core. The diffusion view explains the update rule and its spectrum; the Lean results describe what the architecture can represent, not what the trained model does. Useful next steps are per-benchmark tuning for all models, more seeds, and an agent-level evaluation large enough to measure task success.

Data and code availability. Code, split manifests, result files, the table and figure generators and the Lean proofs: https://github.com/cap-jmk-real/nca_research (MIT licence). Every number in this paper is a macro written by the two table generators in `scripts/` from the result files listed in Table 5. Checkpoints too large for git are listed with SHA-256 hashes in the artifact manifest in `stage0/stage0_results/e1v2/`.

Use of AI tools. AI coding and writing assistants were used to write code, run experiments and draft text. The author checked the result files and is responsible for the content.

Competing interests and funding. None.

References

[1] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin. Growing neural cellular automata. *Distill*, 5(2):e23, 2020. https://doi.org/10.23915/distill.00023

[2] D. Grattarola, L. Livi, and C. Alippi. Learning graph cellular automata. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2110.14237.

[3] G. Gala, D. Grattarola, and E. Quaeghebeur. E(n)-equivariant graph neural cellular automata. arXiv:2301.10497, 2023.

[4] M. Etcheverry et al. Reasoning with neural cellular automata. arXiv:2609.36126, 2026.

[5] L. Pio-Lopez et al. BraiNCA: brain-inspired neural cellular automata and applications to morphogenesis and motor control. arXiv:2604.01932, 2026.

[6] A. Mittal et al. TextNCA: Neural cellular automata for language modeling via hierarchical local attention. arXiv:2608.02050, 2026.

[7] D. Lee et al. Training language models via neural cellular automata. arXiv:2603.10055, 2026.

[8] P. Kontschieder, M. Fiterau, A. Criminisi, and S. Rota Bulò. Deep neural decision forests. In *Proc. ICCV*, 2015. https://doi.org/10.1109/ICCV.2015.172

[9] N. Frosst and G. Hinton. Distilling a neural network into a soft decision tree. arXiv:1711.09784, 2017.

[10] Y. Yang, I. G. Morillo, and T. M. Hospedales. Deep neural decision trees. arXiv:1806.06988, 2018.

[11] L. Chen, M. Zaharia, and J. Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. arXiv:2305.05176, 2023.

[12] I. Ong et al. RouteLLM: Learning to route LLMs with preference data. arXiv:2406.18665, 2024.

[13] S. Singla et al. AgentWeave: Routing before reasoning for efficient function calling in tool-rich language models. arXiv:2608.23078, 2026.

[14] J. Li. Fast models, slow evidence: A paired and self-audited evaluation of System-1 decision models for LLM agent harnesses. arXiv:2610.02267, 2026.

[15] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez. Gorilla: Large language model connected with massive APIs. arXiv:2305.15334, 2023.

[16] S. G. Patil et al. The Berkeley Function Calling Leaderboard (BFCL), dataset. https://huggingface.co/datasets/gorilla-llm/Berkeley-Function-Calling-Leaderboard, 2024.

[17] Y. Qin et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *Proc. ICLR*, 2024. arXiv:2307.16789.

[18] M. Li et al. API-Bank: A comprehensive benchmark for tool-augmented LLMs. In *Proc. EMNLP*, 2023. arXiv:2304.08244.

[19] Hermes agent reasoning traces (filtered). https://huggingface.co/datasets/DJLougen/hermes-agent-traces-filtered, derived from https://huggingface.co/datasets/lambda/hermes-agent-reasoning-traces, accessed 2026.

[20] F. R. K. Chung. *Spectral Graph Theory*. CBMS Regional Conference Series in Mathematics 92, American Mathematical Society, 1997. https://doi.org/10.1090/cbms/092

[21] G. Katz et al. The Marabou framework for verification and analysis of deep neural networks. In *Proc. CAV*, LNCS 11561, pp. 443–452, 2019. https://doi.org/10.1007/978-3-030-25540-4_26

[22] S. Wang, H. Zhang, K. Xu, X. Lin, S. Jana, C.-J. Hsieh, and J. Z. Kolter. Beta-CROWN: Efficient bound propagation with per-neuron split constraints for complete and incomplete neural network robustness verification. In *NeurIPS*, 2021. arXiv:2103.06624.

A. Lean Axiom Audit

Output of `#print axioms` for every top-level theorem (`stage0/stage0_results/review_1110/lean_axioms.txt`), generated with `lake env lean` in `formal_proofs/`. "Standard" axioms are Lean's `propext`, `Quot.sound` and `Classical.choice`.

```
'NCAMetaLearning.fixed_point_unique' depends on axioms: [propext, Classical.choice, Quot.sound]
'NCAMetaLearning.trajectory_decay' depends on axioms: [propext, Quot.sound]
'NCAMetaLearning.cellularTreeStep_preserves_eval' does not depend on any axioms
'NCAMetaLearning.iterate_cellular_tree_preserves_eval' does not depend on any axioms
'NCAMetaLearning.cellular_tree_converges_in_depth' depends on axioms: [propext, Classical.choice, Quot.sound]
'NCAMetaLearning.tree_automaton_edges_in_gnca' does not depend on any axioms
'NCAMetaLearning.powTwo_ge_succ' depends on axioms: [propext, Quot.sound]
'NCAMetaLearning.decision_tree_cellular_duality' depends on axioms: [propext, Classical.choice, Quot.sound]
'NCAMetaLearning.permuteAdj_symm' does not depend on any axioms
'NCAMetaLearning.neighborhood_equivariance' does not depend on any axioms
'NCAMetaLearning.path_equivariance' does not depend on any axioms
'NCAMetaLearning.lineGraphAdj_symm' does not depend on any axioms
'NCAMetaLearning.starGraphAdj_symm' does not depend on any axioms
'NCAMetaLearning.theorem_subsumption_1d' does not depend on any axioms
'NCAMetaLearning.star_graph_diameter_two' does not depend on any axioms
'NCAMetaLearning.line_graph_distance_bound' depends on axioms: [propext, Classical.choice, Quot.sound]
'NCAMetaLearning.theorem_receptive_field_separation' depends on axioms: [propext, Classical.choice, Quot.sound]
'NCAMetaLearning.treeGraphAdj_symm' does not depend on any axioms
'NCAMetaLearning.theorem_tree_subsumption' does not depend on any axioms
'NCAMetaLearning.dyadic_of_scaled_lip' depends on axioms: [propext]
'NCAMetaLearning.trained_damped_recurrence_is_dyadic_contraction' depends on axioms: [propext]
'NCAMetaLearning.trained_damped_recurrence_perturbation_decay' depends on axioms: [propext, Quot.sound]
'NCAMetaLearning.weighted_sum_bounded_two' depends on axioms: [propext]
'NCAMetaLearning.weighted_sum_bounded_three' depends on axioms: [propext, Quot.sound]
'NCAMetaLearning.call_tree_parent_is_ancestor' depends on axioms: [propext, Quot.sound]
'NCAMetaLearning.call_tree_parent_adj' does not depend on any axioms
'NCAMetaLearning.sigmoidal_gating_positive_margin_concordance' depends on axioms: [propext, Quot.sound]
'NCAMetaLearning.sigmoidal_gating_negative_margin_concordance' depends on axioms: [propext, Quot.sound]
Lean (version 4.35.0, x86_64-unknown-linux-gnu, commit eb6cafb7290e8bd5f2937522d8d804f58f02bafc, Release)
```

Theorem 2 is `decision_tree_cellular_duality` in the directory `formal_proofs/`.

B. Perturbation Bound

Proposition 3. *Let $\tilde{T}$ be $\tilde{L}$-Lipschitz on a complete normed space with $\tilde{L} < 1$ and fixed point $\tilde{h}^*$, and let T satisfy $\|T(h) - \tilde{T}(h)\| \le \varepsilon$ for all h. Then the iterates $h^{(t+1)} = T(h^{(t)})$ satisfy*

$$\|h^{(t)} - \tilde{h}^*\| \le \tilde{L}^t \|h^{(0)} - \tilde{h}^*\| + \varepsilon \frac{1 - \tilde{L}^t}{1 - \tilde{L}},$$

$$\limsup_{t \to \infty} \|h^{(t)} - \tilde{h}^*\| \le \frac{\varepsilon}{1 - \tilde{L}}.$$

Proof. Let $e_t = \|h^{(t)} - \tilde{h}^*\|$. Since $\tilde{h}^* = \tilde{T}(\tilde{h}^*)$, $e_{t+1} \le \|T(h^{(t)}) - \tilde{T}(h^{(t)})\| + \|\tilde{T}(h^{(t)}) - \tilde{T}(\tilde{h}^*)\| \le \varepsilon + \tilde{L} e_t$. Unrolling gives $e_t \le \tilde{L}^t e_0 + \varepsilon \sum_{s=0}^{t-1} \tilde{L}^s$, which is the bound for finite t; letting $t \to \infty$ with $\tilde{L}^t \to 0$ gives the limit. □

For the GNCA, T is one step of Eq. (4). The bound would apply if a contractive surrogate $\tilde{T}$ were found within ε of the trained step. Section 4 shows that the trained step itself has Jacobian norm above one at every sampled state; we know of no such surrogate and do not claim one.

C. Result Sources and Changes from Version 7.6

Table 5 names the result sources used in this paper. Version 7.6 contained claims that no result file supports, or that the code contradicts. Table 6 lists what was removed or replaced. The repository file `paper/LEDGER_V76C_1110.md` gives the exact file and key for every entry.

Table 5: Result sources. Directories are relative to `stage0/stage0_results/`.

Id	Directory	Content
S1	`review_1110/`	Four-benchmark accuracies, per-turn correctness, paired statistics, input-matched trees, CPU cost, Jacobian audit, Lean axiom log
S2	top level	Hermes factorial run (reviewer suite): depth, topology, loss, carry, halting, contractive variant, escalation, learning curve
S3	top level	Earlier full-history Hermes run with a different tree-model feature pipeline
S4	`e1v2/run/`	BFCL long-training run (150,000 steps)
S5	`baselines_20261026/`	Local LLM routing baselines (Qwen3.5-9B, Gemma-4-12B)

Table 6: Ledger of Version 7.6 claims and their replacements (sources: Table 5).

Version 7.6 claim	Action	Replacement and source
Title "From Grokking to . . . "	Removed	Gradual learning curve, 68.15 → 70.79% best-so-far validation (S2)
Headline K=8 GNCA (68.92%) and "statistically non-inferior" to HistGradientBoosting (TOST)	Removed	Pre-specified K=4 tree; difference tests only (Table 3, S1)
Per-turn bootstrap over seeds × turns, Cohen's d/h, power 0.94, linear mixed-effects +3.16 pp	Replaced	Seed-clustered paired bootstrap, Holm, DerSimonian–Laird (S1)
"Pre-registered" confirmatory claims	Removed	No pre-registration exists; primary family stated in Section 5
50-trial Hyperband tuning of all models	Removed	Fixed defaults in the scripts; no search was run
Escalation weight β=0.5, "calibrated" score with range $[0, 1.50]$, 10-bin reliability audit	Replaced	β=1 as in the evaluation script; AUROC and two operating points (S2); no calibration result exists
Damped update with α_d, γ and Bernoulli(p=0.8) cell masks	Replaced	Update rule of the model code (Eq. (4))
Causal lower-triangular adjacency	Replaced	Undirected row-normalised `build_adjacency` (Section 3.2)
$\alpha_{\text{self}} \times K$ grid; "ablation" and "Pareto" figures	Removed (values hard-coded in plot scripts)	Factorial table (Table 4, S2)
Contractive variant "satisfies the dyadic premise on 100% of turns", $\rho \leq 0.47$, ratio max 0.437; "$L_{\text{deploy}} \leq 0.8$"	Removed	The audit keys are not written by any repository code; accuracy 61.99% kept (S2); Jacobian audit (S1)
Bridge constants (Jacobian 0.887/0.890, 62,645 turns, radius 5.14, 1.64)	Removed	No result file
"Ten theorems with zero axioms"	Replaced	28 theorems, 13 axiom-free, rest standard axioms (S1)
Tree concordance 76.63%, $\kappa = 0.5628$, as evidence for the tree-simulation theorem	Removed	The value is in a top-level audit file, but no repository script produces it, and agreement with a separately fitted tree does not test the theorem
GCN, LGCA, E(n)-GNCA, Translucent, TextNCA, ARTS, Contriever, Hammer baselines	Removed	No results in the repository; prior work cited in Section 2
Transformer "1.2M parameters, 65.87%, 3.8 ms"	Removed	An earlier-protocol Transformer (276,749 parameters, 65.87 ± 2.00%) is in S5, but it was not run on the four-benchmark protocol; the 1.2M and 3.8 ms figures have no file
Qwen2.5-1.5B constrained decoding 55.20%; frontier-API latencies	Removed	Local LLM numbers kept: Qwen 35.01%/378 ms, Gemma 38.02%/848 ms (S5)
Linear classifier 60.71%; kFLOPs, RAM 142.6 MB	Replaced	Logistic regression row (S1); serialised sizes (S1)
Synthetic 10k-trajectory and probe-leakage numbers	Removed	No result file; leakage handled analytically (Proposition 2)
Intel Xeon 8375C latency (21.2 ms, 60.6×), multi-hardware audit, 18.2M Random-Forest nodes	Replaced	AMD Ryzen 9 7950X 16-Core Processor latencies, 307,118 nodes (S1)
Hermes HistGradientBoosting 69.47% as the main comparison	Kept as sensitivity	S3; main table from S1
Parameter breakdown with inconsistent totals (8,846 invariant part)	Replaced	Eq. (5), asserted against S1 by the table generator
Hugging Face model link; references without a verifiable record	Removed	Repository link; bibliography checked against arXiv and Crossref